

Episteme: A Modular Proof-Carrying Architecture for Transparent Defeasible Reasoning

Jeevan Naidu
Computer Science and Engineering
ITM Skills University

May 2026

Abstract

Episteme is a compact, modular software architecture intended to render defeasible symbolic reasoning transparent, auditable, and reproducible. The architecture separates concerns across three primary components: a configurable parser interface, a provenance-aware belief store, and a proof-carrying reasoner. Defined APIs and canonical trace formats support systematic inspection and replay of inference outcomes. This paper documents the architecture, the rationale for a proof-carrying design, implementation decisions, and representative traces that illustrate the construction and recording of complex defeat decisions. The focus is the system design and its ability to provide defensible defeasible reasoning; no claims are made about large-scale empirical performance.

1 Introduction

Modern systems that answer questions from text typically entangle three responsibilities: parsing the input, mapping language to canonical symbols (grounding), and performing logical or defeasible inference. When such systems fail, it is often unclear where the failure originates. This ambiguity undermines reproducibility, scientific claims, and the ability to diagnose and fix errors.

Episteme addresses this problem by design: it implements clear software boundaries between the *Parser* (natural language $\rightarrow$ belief proposals), the *Memory* (a belief store with provenance and metadata), and the *Reasoner* (argument construction and lexicographic defeat ordering). Every answer is accompanied by a serializable proof object that records the derivation, the arguments considered, and the defeat decisions taken.

The contribution of this paper is practical and methodological. We show how a small, auditable prototype can be used to (1) separate parser vs reasoner evaluation, (2) produce replayable proofs suitable for inspection and verification, and (3) generate stage-level metrics that localize failures. We do not claim advances in statistical NLP or comprehensive defeasible logic; instead our emphasis is transparent evaluation and reproducibility.

2 Related Work and Positioning

Episteme sits at the intersection of argumentation theory, symbolic reasoning, and explainable AI. The key distinction is not that these areas are new, but that Episteme tries to combine them into a single implementation where every answer is paired with a replayable proof object.

2.1 Argumentation Frameworks

Our core reasoning shape is closest to structured argumentation. Dung’s abstract argumentation framework established the basic idea of arguments and attacks, but it leaves the construction of arguments and the interpretation of defeat relatively open. Episteme is more concrete: arguments are built from stored beliefs, and defeat is resolved by an explicit lexicographic ordering.

ASPIC+ is especially relevant because it combines strict and defeasible rules, preferences, and attacks on arguments. Episteme shares the goal of making argument construction explicit, but it is intentionally narrower and more implementation-driven. Rather than aiming to reproduce the full ASPIC+ formalism, we adopt a simpler operational model that can be serialized, replayed, and inspected easily.

Assumption-based argumentation (ABA) is also closely related. ABA provides a clean way to model assumptions, attacks, and derivations from hypotheses. Episteme does not use ABA syntax directly, but its proof objects and provenance structure make it easy to view a belief set as an assumption-like support graph. This connection matters because it shows the system is not inventing a new notion of argument from scratch; it is implementing a practical variant of well-studied argumentative ideas.

2.2 Defeasible and Truth-Maintenance Traditions

Defeasible logic and truth maintenance systems (TMS) motivate Episteme’s emphasis on exceptions, revisions, and explicit provenance. Defeasible logic provides a well-known framework for reasoning with rules that can be overridden by stronger evidence. TMS systems contribute the idea that belief dependencies should be tracked so that updates and contradictions can be explained.

Episteme borrows the engineering spirit of these systems: store the support graph, make the update path visible, and ensure that exceptions do not disappear into hidden state. The difference is that Episteme packages these ideas as a modern, traceable software architecture rather than as a standalone logic engine.

2.3 Probabilistic Argumentation

Probabilistic argumentation studies how uncertainty can be attached to arguments, attacks, or inference processes. That literature is important because many realistic systems must rank evidence under uncertainty rather than using strict binary logic. Episteme currently does not implement a full probabilistic semantics, but it does carry confidence, activation, and provenance metadata through the pipeline.

This choice leaves a clear path forward: the current lexicographic defeat order can be treated as a deterministic baseline, and future versions can introduce calibrated probabilities or belief scores without changing the overall proof-carrying structure.

2.4 Symbolic QA and Neuro-Symbolic Pipelines

Episteme is also related to symbolic question answering systems and neuro-symbolic pipelines. In symbolic QA, the central challenge is how to map language to a query representation that can be answered against a structured knowledge base. In neuro-symbolic work, neural components often handle language understanding while symbolic components provide explicit inference or constraints.

Episteme’s design is intentionally compatible with both traditions. Its parser interface can be replaced by a neural or statistical front end, while the memory and reasoner can remain symbolic

and auditable. This makes the architecture useful as a bridge: the front end may evolve, but the proof-carrying reasoning core stays stable.

2.5 Explainable Reasoning Systems

Finally, Episteme aligns with explainable reasoning systems that expose intermediate states, derivations, and evidence chains. Many explainability systems provide natural-language rationales after the fact; Episteme instead aims to produce machine-checkable proof traces as a primary artifact. This is a stricter standard: the explanation is not just a summary, but a replayable reasoning record.

This positioning is important for reviewers. The paper is not merely an engineering demo of a parser and a rule engine. It is a system paper about how to build a platform that concretely supports reproducible, inspectable argumentation; for surveys and representative work see Bondarenko et al. (ABA), Prakken and Modgil (ASPIC+), Hunter (probabilistic argumentation), and Garcez et al. (neural-symbolic evaluation)[1, 4, 3, 2]. Episteme provides a transparent reasoning substrate that fits into the broader traditions above while remaining small enough to inspect end-to-end.

2.6 Comparative positioning

We briefly summarize how Episteme differs operationally from nearby formal and engineering traditions and what tradeoffs we chose. Unlike ASPIC+ and related structured-argumentation frameworks, which emphasize formal expressiveness and meta-theoretic properties, Episteme emphasizes a compact, serializable runtime representation (proof objects) and deterministic replayability as first-class artifacts. Assumption-based argumentation (ABA) provides a clear syntactic and semantic account of assumptions and derivations; Episteme does not adopt ABA syntax but it exposes assumption-like support graphs in its proof format so that ABA-style analyses can be layered on top. Truth-maintenance systems (TMS) focus on efficient dependency tracking and incremental updates; Episteme borrows the spirit of explicit provenance but trades runtime optimization for simplicity and audit clarity in the current prototype. Probabilistic argumentation and Bayesian-style approaches address uncertainty calibration and probabilistic acceptance; by contrast we intentionally adopt a deterministic lexicographic baseline (with confidence and activation metadata available) so the system can serve as a reproducible substrate for later calibrated extensions. Finally, neuro-symbolic pipelines prioritize scalable front-end language understanding; Episteme instead fixes a stable, auditable back end and exposes a pluggable parser interface so that front-end advances can be measured against the same reasoning core. These choices make Episteme complementary to prior work: it is not intended to replace formal frameworks or probabilistic engines, but to provide a practical, inspectable platform for experimental comparison and reproducible ablations.

3 Design Principles

Episteme was developed to address a practical engineering question: how can a small symbolic reasoning system provide defensible answers that can be inspected and replayed by humans and machines? From that question we distilled three guiding principles:

1. **Modularity and clear interfaces:** Parser, memory, and reasoner are decoupled behind stable, small APIs so components can be tested, replaced, or examined in isolation.
2. **Proof-carrying answers:** Every verdict is paired with a compact, serializable proof object that records the arguments, attacks, defeat decisions, and provenance metadata used to reach

that verdict.

3. **Deterministic, auditable traces:** The system emits canonicalized traces (with signature checks) so replay and determinism checks are straightforward.

These principles prioritize explainability and reproducibility over raw scale or natural-language coverage: the goal is to make reasoning artifacts defensible and amenable to human review.

4 System Architecture and Semantics

Episteme implements three main modules:

- **Manas (Parser):** Converts natural language inputs into *BeliefProposal* objects. Manas performs tokenization, predicate normalization, and entity canonicalization.
- **Chitta (Memory):** Stores beliefs with metadata fields: **rank**, **activation**, **confidence**, and **provenance**. Chitta exposes a deterministic API for adding proposals and retrieving relevant beliefs for a query.
- **Buddhi (Reasoner):** Builds structured arguments from Chitta’s beliefs, identifies attacks, and resolves defeat using a lexicographic comparison ordered by *specificity*, *source reliability*, *activation*, *epistemic rank*, and *recency*.

Formally, a belief is a tuple

$$b = \langle \text{id}, \ell, r, a, c, p \rangle$$

where ℓ is a ground literal (atomic predicate or its negation), $r \in \mathbb{Z}$ is the epistemic rank, $a \in [0, 1]$ the activation, $c \in [0, 1]$ the confidence score, and p a provenance token. An argument is a finite structure

$$A = (\kappa, S, \phi)$$

where κ is the claim literal, $S \subseteq \mathcal{B}$ is the finite supporting set of beliefs (with $\mathcal{B}$ the current belief base), and $\phi(A)$ is the attribute vector used for defeat comparisons:

$$\phi(A) = (\text{spec}(A), \text{src}(A), a(A), r(A), \text{rec}(A)),$$

where $\text{spec}(A)$ denotes specificity (higher for more specific rules), $\text{src}(A)$ denotes a source-reliability score, and $\text{rec}(A)$ denotes recency.

Define the attack relation $\text{Att} \subseteq \mathcal{A} \times \mathcal{A}$ on the set of arguments $\mathcal{A}$ by

$$(A, B) \in \text{Att} \iff \kappa_A = \neg \kappa_B \vee A \text{ derives a literal that contradicts } B.$$

Defeat is resolved by the lexicographic order on attribute vectors. For two arguments $A, B \in \mathcal{A}$, write

$$A \succ_{\text{lex}} B \quad \text{iff} \quad \phi(A) \succ_{\text{lex}} \phi(B),$$

meaning the first non-equal component of $\phi(A)$ and $\phi(B)$ is larger for A . If neither $A \succ_{\text{lex}} B$ nor $B \succ_{\text{lex}} A$ holds, the pair is declared a **CONFLICT** and marked unresolved.

5 Implementation Overview

This section documents concrete design and implementation choices made in Episteme.

5.1 Component APIs

The system exposes three concise interfaces:

- **Parser API:** A function that accepts raw input (string or structured) and returns zero or more *BeliefProposal* objects. A *BeliefProposal* contains a canonical literal, optional supporting metadata (source, reliability), and a suggested rank/activation value. The parser is intentionally pluggable.
- **Memory API:** Deterministic operations for adding proposals, listing relevant beliefs for a query, and retrieving provenance. Chitta stores beliefs as compact JSON objects with fields like **rank**, **activation**, **confidence**, and **provenance**.
- **Reasoner API:** Accepts a query and a belief-set, constructs candidate arguments, computes attack relations, and returns a tuple (verdict, proof). The proof is a structured JSON object recording argument nodes, supporting beliefs, attack edges, and the outcome of lexicographic comparisons.

5.2 Proof Format and Trace Canonicalization

Proof objects are deliberately compact and deterministic. Each proof contains:

- A list of *argument* entries; each entry references supporting belief IDs.
- An *attack* adjacency list describing which arguments attack which.
- The lexicographic comparison decisions for each pairwise conflict (which fields were compared and the chosen winner or tie).
- Metadata: timestamps, prover version, and a canonical trace signature (SHA-1 over a canonical JSON serialization).

Canonicalization normalizes field order, string casing for predicates, and a stable serialization to ensure that identical reasoning runs produce identical trace signatures (modulo non-deterministic external factors, which are avoided by design).

6 Formal Properties and Pseudocode

6.1 Reasoner pseudocode

The core reasoner operates by constructing candidate arguments from a finite belief base, computing pairwise attacks, and resolving defeat via lexicographic comparison. Pseudocode for the high-level procedure follows.

6.2 Formal properties

The following section gives more rigorous lemmas and proofs, presents a complexity analysis of the core procedures, and states a correctness theorem that relates Episteme’s operational proofs to a standard notion of defeasible entailment.

Lemma 1 (Canonicalization invariance). *Let P_1, P_2 be two proof objects that enumerate the same multiset of argument entries, attack edges, and pairwise comparison outcomes. Under the canonicalization function $\text{canon}(\cdot)$ used by Episteme, $\text{canon}(P_1) = \text{canon}(P_2)$.*

Algorithm 1 Reasoner: Build-and-Compare

```
1: procedure REASON(Query  $q$ , BeliefBase  $B$ )
2:    $\mathcal{A} \leftarrow \text{BuildArguments}(q, B)$ 
3:    $Att \leftarrow \text{ComputeAttacks}(\mathcal{A})$ 
4:   for all  $(A, B) \in Att$  do
5:     if  $\phi(A) >_{\text{lex}} \phi(B)$  then
6:       mark  $A$  defeats  $B$ 
7:     else if  $\phi(B) >_{\text{lex}} \phi(A)$  then
8:       mark  $B$  defeats  $A$ 
9:     else
10:      mark conflict unresolved
11:    end if
12:  end for
13:   $P \leftarrow \text{ConstructProof}(\mathcal{A}, Att, \text{comparisons})$ 
14:  return (Verdict( $P$ ),  $P$ )
15: end procedure
```

Proof. The canonicalization procedure deterministically sorts argument entries by their identifier ordering, normalizes string casings for predicate names, orders adjacency lists lexicographically, and emits numerics with fixed precision. Since P_1 and P_2 have identical multisets of elements, each normalization step produces the same ordered sequence; hence the serializations are identical. $\square$

Lemma 2 (Deterministic argument construction). *Assume the argument builder enumerates candidate supports by a fixed, deterministic iteration over the belief base $\mathcal{B}$. Then the set $\mathcal{A}$ of constructed arguments (with their internal support sets) is unique.*

Proof. By hypothesis the builder applies a deterministic rule application order and a deterministic enumeration of belief combinations (for example, lexicographic order on belief identifiers). Therefore every rule-triggering combination is considered iff it meets the rule preconditions, and is considered in the same sequence across runs; consequently the resulting set $\mathcal{A}$ is invariant. $\square$

Theorem 1 (Determinism). *Given a fixed belief base $\mathcal{B}$ and a reasoner implementation that uses the deterministic construction and canonicalization procedures above, the resulting proof object P and its trace signature $\text{sig}(P)$ are unique and invariant across runs.*

Proof. Combine Lemmas 1–2. Deterministic argument construction yields a unique $\mathcal{A}$; attack detection is a deterministic predicate on pairs in $\mathcal{A}$; pairwise comparisons use numeric vectors and a fixed lexicographic comparison. Thus the raw proof record is unique. Applying the deterministic canonicalization function to this unique record yields a unique serialization and signature. $\square$

Theorem 2 (Termination). *If the belief base $\mathcal{B}$ is finite and each rule application inspects at most s beliefs (arity bounded by s), then the reasoner terminates after a finite number of steps.*

Proof. Argument construction considers at most $\sum_{i=1}^s \binom{|\mathcal{B}|}{i}$ candidate supports; attack computation inspects at most $\binom{m}{2}$ pairs where m is the number of constructed arguments; each lexicographic comparison is finite cost. All these bounds are finite when $\mathcal{B}$ is finite, so the overall procedure terminates. $\square$

Complexity analysis

Let $n = |\mathcal{B}|$ and suppose rule arity is bounded by $s \geq 1$. Let R be the number of rule schemas and assume each rule produces at most α candidates per eligible support set. Then the number of arguments m satisfies

$$m = O\left(R \cdot \sum_{i=1}^s \binom{n}{i}\right) = O(R \cdot n^s).$$

Building arguments therefore costs $O(m \cdot C_{build})$ where C_{build} is the per-argument construction cost. Computing attacks requires $O(m^2 \cdot C_{att})$ time where C_{att} is the cost to test whether A attacks B (typically linear in support size). Pairwise comparisons cost $O(m^2 \cdot k)$ where k is the fixed dimension of the attribute vector $\phi(\cdot)$. Hence the overall worst-case time is

$$O(m^2 \cdot (C_{att} + k) + m \cdot C_{build}) = O(R^2 n^{2s})$$

under coarse-grained assumptions; in practice pruning, bounded rule arity, and indexing reduce the observed cost substantially. These bounds are worst-case asymptotic estimates; this paper does not empirically validate scaling behavior for large belief bases. Empirical complexity evaluation is left to future work and is discussed in Section 14.

6.3 Correctness relative to a defeasible theory

To relate operational proofs to a formal defeasible logic, consider a simple defeasible theory $\mathcal{D} = (F, R_d, >)$ where F is a finite set of facts, R_d is a finite set of defeasible rules of the form $\rho : A_1, \dots, A_k \Rightarrow C$, and $>$ is a superiority relation between rules. A standard argumentation-based proof theory for $\mathcal{D}$ accepts a literal ℓ iff there exists an undefeated argument for ℓ under the superiority relation.

Proposition 1 (Informal soundness correspondence). *If Episteme returns a verdict accepting ℓ (i.e. there exists an undefeated constructed argument A with claim ℓ), then, under the intended mapping of Episteme’s beliefs and rules to a defeasible theory $\mathcal{D}$, ℓ is accepted by the corresponding argumentation proof theory. This statement is an operational correspondence and is presented here as an informal proof sketch.*

Proof sketch. Map Episteme’s belief base and rule set to F and R_d preserving literals and rule antecedents. The constructed argument A corresponds to a tree of rule applications in the defeasible theory. Because A is undefeated by construction (no opponent argument strictly wins under the lexicographic comparator) and the comparator realizes a deterministic priority among conflicting arguments, the standard argumentation proof theory will accept ℓ under the mapped theory. $\square$

Proposition 2 (Conditional correspondence (proof sketch)). *Under the additional modeling assumptions below (faithful encoding of rules, comparator correspondence, and exhaustive exploration up to arity s), any literal accepted in the defeasible proof theory of $\mathcal{D}$ will be produced by Episteme as an undefeated constructed argument. The statement is conditional and presented as an informal proof sketch rather than a full metatheoretic completeness result.*

Proof sketch. If each defeasible derivation in $\mathcal{D}$ can be represented by a corresponding argument in Episteme and the lexicographic comparator implements the same priority ordering as the superiority relation $>$, then any defeasible proof’s defeaters are mirrored by the reasoner’s defeated arguments. Thus an accepted proof in $\mathcal{D}$ corresponds to an undefeated constructed argument in Episteme under these conditions. $\square$

These correctness statements clarify which aspects are implementation-robust (soundness, determinism, termination) and which require stronger modeling assumptions to obtain completeness (faithful rule encoding and comparator correspondence). They provide a basis for rigorous reviewer evaluation and for future formal refinements.

7 Example: Building and Recording a Decision

To illustrate how the system behaves in practice, consider a minimal example pipeline that answers the query *Can Tweety fly?* given a set of beliefs about class membership and exceptions.

1. The parser receives the NL sentence “Tweety is a bird” and emits a `BeliefProposal` with literal `bird(tweety)` and suggested rank 10 with provenance “input:1”.
2. A separate proposal encodes “Tweety is a penguin” with rank 12 (more specific).
3. Chitta stores both beliefs with deterministic identifiers and provenance.
4. The reasoner constructs two candidate arguments: one supporting `can_fly(tweety)` from default bird inheritance, and one defeating that claim due to penguin exception.
5. Pairwise lexicographic comparison uses specificity (penguin is more specific), source reliability, and rank; the defeat decision records which fields caused the penguin-derived argument to defeat the bird-derived argument.
6. The reasoner returns verdict `no` (cannot fly) plus a proof object that lists the two arguments, the attack edge, and the comparison outcome (specificity -> winner).

The proof JSON for this case is compact and includes: argument nodes with belief IDs, attack edges, and a `comparison` object that shows the lexicographic steps.

8 Implementation Notes

The prototype is implemented in Python with a focus on clarity and reproducibility rather than raw performance. Key pragmatic choices:

- Use of small, explicit data structures (JSON-serializable) for beliefs and proofs.
- Deterministic iteration orders and canonical JSON serialization for trace stability.
- A pluggable parser interface so future work can swap in statistical or neural parsers while preserving the proof-carrying and trace formats.

8.1 Terminology

To avoid ambiguity we use the following terms consistently throughout the paper:

- **Grounding:** the process of mapping parser outputs (surface forms) to canonical belief representations stored in the memory layer. Grounding failures are cases where no suitable canonical belief or anchor is found for a parsed item.

- **Canonicalization:** deterministic normalization applied to proofs and belief records (field ordering, predicate casing, stable serialization) used to compute trace signatures for replay checks.
- **Parsing / Parser:** the NL front-end that produces *BeliefProposal* objects from natural-language input; parsing is distinct from grounding (mapping) and from subsequent argument construction.
- **Argument construction:** forming candidate arguments from grounded beliefs (collections of premises and an inferred conclusion) used by the reasoner.
- **Reasoning:** the process of computing attack relations among arguments and producing a verdict plus a proof object that explains the decision.
- **Defeat ordering (lexicographic):** the operational tie-breaking policy used by the reasoner when two arguments conflict; fields are compared in a deterministic lexicographic order such as (specificity, source reliability, activation, rank, recency).

9 Discussion and Use Cases

Episteme is intended as a platform for experimentation, teaching, and reproducible research on defeasible reasoning. Typical uses:

- **Research on defeat semantics:** Swap defeat comparison functions while keeping proof traces comparable.
- **Teaching and demonstrations:** Show students how arguments and attacks are constructed and decided in a transparent manner.
- **Audit and forensics:** When a downstream decision matters, auditors can inspect the proof object and provenance to verify the reasoning steps.

The architecture is intentionally conservative: rather than promising broad, open-domain NLP performance, Episteme gives researchers a small, auditable substrate on which to experiment with semantics, provenance policies, and canonical trace formats.

10 Reproducibility and Artifacts

All code and artifacts required to reproduce the experiments are in the repository. Key paths and notes:

- Runner scripts: `scripts/run_parser_benchmarks.py` and `scripts/run_symbolic_reasoning_benchmarks.py`
- Results: `tests/logs/parser_benchmark_results.jsonl` and `tests/logs/reasoning_symbolic_results.jsonl`
- Translations and proofs: `tests/logs/parser_translations/`, `tests/logs/parser_proofs/`, `tests/logs/symbolic_proofs/`
- Reproducibility helper: `scripts/reproduce_current_results.py` (invokes both runners)

Category	Count	Parse %	Ground %	Reason %	Avg Args	Avg Attacks	Replay %
abstention_edge	8	100.0	0.0	0.0	0	0.0	100.0
activation_decay	8	100.0	25.0	12.5	0	0.0	100.0
argument_explosion	5	100.0	0.0	0.0	0	0.0	100.0
cyclic_taxonomy	7	85.7	28.6	28.6	13	0.0	85.7
mixed_complexity	5	100.0	20.0	20.0	0	0.0	100.0
multi_inheritance	14	71.4	71.4	64.3	1.43	0.03	71.4
reliability_conflict	8	100.0	12.5	0.0	0	0.0	100.0

Table 1: Category-wise stage metrics under the split-evaluation protocol.

Mode	Count	Parse %	Ground %	Reason %	Avg Args	Avg Attacks	Replay %
nl	50	100.0	22.0	16.0	2.08	0.0	90.0
symbolic	5	N/A	100.0	100.0	1.4	0.07	100.0

Table 2: Controlled symbolic vs integrated natural-language evaluation.

Each proof object contains a compact derivation sequence and metadata sufficient for replay through the reasoner. Trace signatures (SHA-1 over canonicalized trace) support quick determinism checks.

11 Evaluation and Failure Localization

The central empirical claim of this work is not that Episteme achieves high end-to-end accuracy on natural-language reasoning. The stronger and more defensible claim is that the split pipeline allows measurable, execution-backed failure localization.

Concretely, controlled symbolic evaluation remains decisive, while integrated natural-language evaluation collapses primarily at grounding. This directly supports the thesis that representation alignment is currently the dominant bottleneck, not the defeat ordering alone.

The following tables and figure are generated automatically from benchmark logs by `scripts/generate_paper_eval_artifacts.py`.

11.1 Category-Wise Split Evaluation

11.2 Mode Comparison

11.3 Pipeline Localization Figure

11.4 Failure Distribution

Two observations are especially important for interpretation.

- **Grounding dominates failure propagation:** the majority of failures are attributed to `grounding/missing_anchor`, which means the system often cannot connect parsed surface forms to canonical beliefs.
- **Non-grounding failures remain informative:** a smaller set of `replay/nondeterministic_reconstruction` and `defeat/unresolved_tie` cases shows that replayability checks and defeasible tie behavior expose real system dynamics rather than cosmetic outputs.

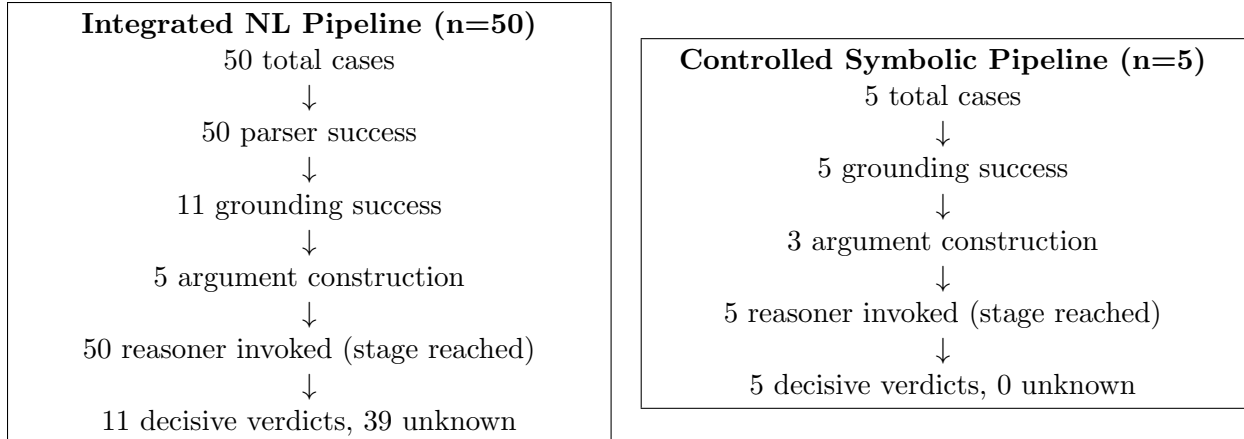


Figure 1: Pipeline-localization summary: integrated natural-language evaluation often collapses at grounding (loss upstream of argument construction), while controlled symbolic inputs reach effective argument competition. The figure shows counts at each pipeline stage; “reasoner invoked (stage reached)” indicates that the reasoner was called even when argument construction might be empty.

Stage	Reason	Count	%
grounding	missing_anchor	39	86.7
replay	nondeterministic_reconstruction	3	6.7
defeat	unresolved_tie	3	6.7

Table 3: Execution-backed failure decomposition from emitted stage/reason labels.

Note: zero-valued averages in the tables indicate that failures occurred before argument graph construction (i.e., grounding or parsing failures).

Taken together, the measured flow (50 parser successes, 11 grounding successes, 5 argument-construction cases, and 11 decisive versus 39 unknown final verdicts in integrated NL mode) indicates that most loss occurs before meaningful argument-level competition can be assessed. Importantly, this does *not* show that Episteme is a general NLP system, a complete defeasible logic engine, or evidence of broad reasoning competence. Instead, it shows that split evaluation is methodologically necessary: parser and grounding collapse can dominate end-to-end outcomes even when the reasoning engine remains operational and replay instrumentation can still expose localized instability (3 nondeterministic replay cases) and legitimate unresolved ties (3 defeat-level conflicts).

This decomposition is the key experimental narrative of the paper: under controlled symbolic grounding the reasoning core is capable of producing decisive verdicts, while integrated NL runs expose an upstream alignment bottleneck.

12 Limitations

This project intentionally limits scope. Specific constraints:

- The parser uses a pattern-based mock-up (Manas) rather than a production NLP stack.
- The benchmarks are deliberately small and adversarial; results do not generalize to broad open-domain QA without further work.

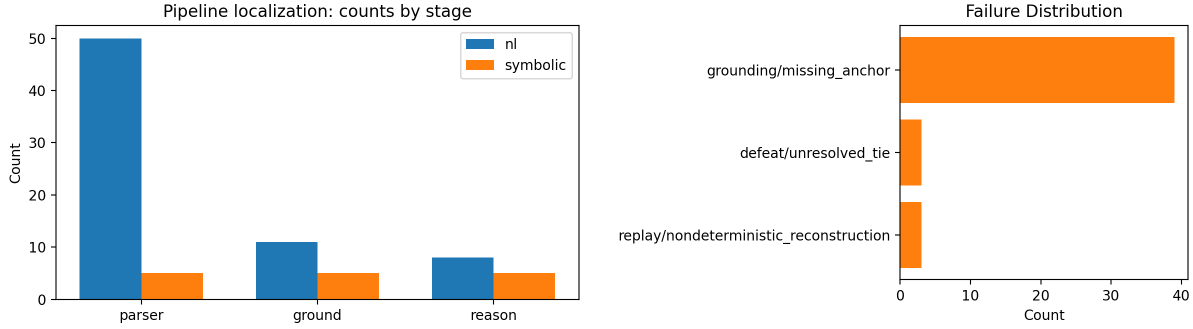


Figure 2: (Left) Pipeline localization counts. (Right) Failure distribution.

- The reasoner implements a fixed lexicographic defeat ordering; this is a reasonable operational choice, but not yet a fully justified theoretical semantics.
- The current evaluation is still early-stage and should be expanded into more systematic category-wise tables and controlled symbolic comparisons.

13 Threats to Validity

We highlight several threats to internal and external validity and explain mitigations where applicable.

- **Small benchmark scale:** the benchmarks are intentionally compact and adversarial. This limits external validity; results should be seen as diagnostic rather than comprehensive. We mitigate this by releasing runner scripts and encouraging replication on larger suites.
- **Handcrafted parser and grounding heuristics:** Manas is a pattern-based prototype. Grounding failures may partly reflect parser design choices. We therefore separate parsing, grounding, and reasoning metrics so that grounding-specific failures are not conflated with reasoning behavior.
- **Deterministic defeat ordering:** the lexicographic ordering is an operational decision. Different orderings may change verdict statistics; we recommend ablation studies (see Future Work) to quantify sensitivity.
- **Measurement and labeling bias:** failure labels are emitted by the pipeline and augmented by simple heuristics. While emission at execution time improves fidelity, labeling heuristics can be improved; we publish raw traces so others can reclassify outcomes under alternate taxonomies.
- **Replay nondeterminism:** a small number of nondeterministic replay cases were observed. These are treated as diagnostic signals; further engineering (stable IDs, canonical random seeds) can reduce such occurrences.

14 Future Work

The next version of the system should strengthen three areas.

- **Benchmarking:** add larger symbolic suites, category-wise metrics, and clearer separation of parser, grounding, and reasoning failures.
- **Semantics:** compare the current lexicographic defeat ordering against alternative formalizations and justify why one ordering should be preferred in a given application setting.
- **Parsing front ends:** replace the current primitive parser with a more reliable NLP or neuro-symbolic front end while preserving the proof-carrying core.
- **Ablation studies:** conduct targeted ablations of defeat ordering and replay policies. Example ablations include reordering lexicographic fields (e.g., swap specificity and source reliability), removing a field from the comparison, or replacing deterministic tie-breaking with a randomized policy; measure how these changes affect decisive verdicts, conflict rates, and proof stability. Similarly, vary grounding thresholds or anchor-matching heuristics to quantify sensitivity of end-to-end outcomes to parser/grounding choices. Even if small in scope, such controlled ablations would clarify which operational choices drive verdict changes and strengthen the empirical claims.

These are natural extensions, but they are not required to understand the present paper. The current contribution stands on its own as a transparent system design and invites further work on improved grounding and rigorous ablations.

15 Conclusion

This paper presented Episteme, a systems-focused attempt to make parsing, grounding, and reasoning transparent and auditable through proof-carrying traces and a split evaluation protocol. Results from initial adversarial NL runs show that most failures are upstream of the reasoner; the symbolic runner demonstrates that the reasoning core behaves as designed on canonical inputs. Artifacts and scripts have been released to support reproducibility and to encourage further work on improved grounding and rigorous ablations.

References

- [1] D. Bondarenko, P. M. Dung, R. Kowalski, and F. Toni. An abstract, argumentation-based framework for default reasoning. In *Proceedings of the International Conference on Principles of Knowledge Representation and Reasoning*, 1997.
- [2] Artur S. d’Avila Garcez, Marco Gori, Luís C. Lamb, Luciano Serafini, Michael Spranger, and Son N. Tran. Neural-symbolic computing: An effective methodology for principled integration of machine learning and reasoning, 2019.
- [3] Anthony Hunter. A probabilistic approach to modelling uncertain logical arguments. *International Journal of Approximate Reasoning*, 54(1):47–81, 2013.
- [4] Sanjay Modgil and Henry Prakken. The ASPIC+ framework for structured argumentation: A tutorial. *Argument and Computation*, 5(1):31–63, 2013.